Aplicaciones Web 1 — Actividad 12

Arreglos y objetos

Agustín Benítez

Consigna

Aplicaciones web 1

Actividad 12

Actividad a desarrollar individualmente como práctica; y la entrega es individual para que quede el registro.
No se permite utilizar la IA para el desarrollo y planificación; solo se utilizará la IA al final para revisar lo hecho.

Ejercicios JS

Desarrollo

- Apoyándose en el TID (SP4)
  https://drive.google.com/file/d/1W_VPRfzbh05xWLZb6pid7BLSm1bTaTMZ/view
- **JavaScript**:
  https://developer.mozilla.org/es/docs/Web/JavaScript

Consola y sintaxis básica

Para realizar el ejercicio, crear un archivo **index.js** y vincularlo a un **index.html**.

1. **Arreglos**

   a. Escriba un programa que solicite a un usuario la carga de 5 números a través de un método **prompt()** que se irán incluyendo en un arreglo con el método **push()**.
   Utilizar un bucle **for** para repetir el proceso 5 veces, y una vez terminado el bucle, imprimir el arreglo en consola.
   Recomendación*: crear el arreglo fuera del bucle primero.*

   b. Modifique el programa del punto anterior, para que incluya un condicional para guardar en el arreglo solo los números impares.

   c. Escriba un programa que contenga un arreglo de nombres de 4 frutas (pera, manzana, banana, durazno), y utilizando el método **forEach()** (ver doc) sobre el

mismo arreglo, convierta a MAYÚSCULAS los nombres (ver documentación). Una vez terminado el proceso, escriba por consola el arreglo.

d. Modifique el programa del punto anterior, y en lugar de utilizar el **forEach()**, utilice el método no destructivo **map()** (ver doc) para crear un nuevo arreglo.

e. Modifique el programa del punto **c**, y en lugar de utilizar el **forEach()**, utilice el método no destructivo **filter()** (ver doc) para crear un nuevo arreglo solo con las frutas que contengan "**na**" en su nombre. Utilice el método de Strings includes() (ver doc) con un condicional.

2. **Objetos**

a. Cree un programa que contenga un objeto con datos ficticios de un producto de indumentaria:
- *nombre*
- *precio*
- *stock*
- *talles: [s,m,l]*

b. Cree un programa que contenga un arreglo de 4 objetos con datos ficticios de productos de indumentaria:
- *nombre*
- *precio*
- *stock*
- *talles: [s,m,l]*

Recorra el arreglo de objetos con **forEach()** e imprima por consola solo los que tienen un stock menor a 5 *(procure tener variedad de stock para que se cumpla la condición en algún momento)*

c. Escriba un programa que re-utilice el arreglo del punto **b.** y use **map()** para crear un nuevo arreglo que contenga los mismos productos pero con un descuento en el precio de un 10%.
Imprima por consola en nuevo arreglo.

d. Escriba un programa que re-utilice el arreglo del punto **b.** y use **filter()** para crear un nuevo arreglo que contenga solo los productos cuyo stock sea mayor a 10 *(procure tener variedad de stock para que se cumpla la condición en algún momento)* .
Imprima por consola en nuevo arreglo.

Resolver sobre un archivo JS, adjuntar consignas en PDF y subir al repositorio de actividades.

Resolución

La resolución está en la carpeta `actividades/actividad-12` del repositorio de actividades:

- `index.html`: página que vincula el script con la etiqueta `<script src="index.js">`.
- `index.js`: resolución de todos los puntos. Cada punto está identificado con un comentario y los resultados se imprimen por consola.

Para verlo, abrir `index.html` en el navegador y mirar la consola (F12 → pestaña *Consola*).

index.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Actividad 12 - Arreglos y objetos</title>
</head>
<body>
  <h1>Actividad 12 - Arreglos y objetos</h1>
  <p>Al abrir la página se piden 10 números con <code>prompt()</code>
(5 para el ejercicio 1.a y 5 para el 1.b).</p>
  <p>Los resultados se muestran en la consola del navegador (F12 →
pestaña Consola).</p>

  <script src="index.js"></script>
</body>
</html>
```

index.js

```
// ACTIVIDAD 12 - Arreglos y objetos
// Cada ejercicio está dentro de un bloque { } para que sus variables
// no se mezclen con las de los otros ejercicios.

// ========================================================
// 1. ARREGLOS
// ========================================================

// a. Cargar 5 números con prompt() y agregarlos al arreglo con push()
console.log("--- 1.a ---");
{
  const numerosIngresados = [];

  for (let i = 1; i <= 5; i++) {
    const numero = Number(prompt(`1.a - Ingresá el número ${i} de
5`));
    numerosIngresados.push(numero);
  }

  console.log(numerosIngresados);
}
```

```
// b. Igual que el anterior, pero guardando solo los números impares
console.log("--- 1.b ---");
{
  const numerosImpares = [];

  for (let i = 1; i <= 5; i++) {
    const numero = Number(prompt(`1.b - Ingresá el número ${i} de 5
(solo se guardan los impares)`));

    if (numero % 2 !== 0) {
      numerosImpares.push(numero);
    }
  }

  console.log(numerosImpares);
}

// c. Convertir a MAYÚSCULAS los nombres de las frutas con forEach()
console.log("--- 1.c ---");
{
  const frutas = ["pera", "manzana", "banana", "durazno"];

  frutas.forEach((fruta, indice) => {
    frutas[indice] = fruta.toUpperCase();
  });

  console.log(frutas);
}

// d. Lo mismo con map(), que crea un nuevo arreglo sin modificar el
original
console.log("--- 1.d ---");
{
  const frutas = ["pera", "manzana", "banana", "durazno"];

  const frutasEnMayusculas = frutas.map((fruta) =>
fruta.toUpperCase());

  console.log(frutasEnMayusculas);
}

// e. Con filter() e includes(): solo las frutas que contienen "na"
console.log("--- 1.e ---");
{
  const frutas = ["pera", "manzana", "banana", "durazno"];

  const frutasConNa = frutas.filter((fruta) => {
    if (fruta.includes("na")) {
```

```
      return true;
    } else {
      return false;
    }
  });

  console.log(frutasConNa);
}

// ======================================================
// 2. OBJETOS
// ======================================================

// a. Objeto con los datos de un producto de indumentaria
console.log("--- 2.a ---");
{
  const producto = {
    nombre: "Remera básica de algodón",
    precio: 12000,
    stock: 25,
    talles: ["s", "m", "l"],
  };

  console.log(producto);
}

// b. Arreglo de 4 productos (se reutiliza en los puntos c y d)
const productos = [
  { nombre: "Remera básica de algodón", precio: 12000, stock: 25,
talles: ["s", "m", "l"] },
  { nombre: "Buzo con capucha", precio: 28000, stock: 3, talles: ["s",
"m", "l"] },
  { nombre: "Jean recto", precio: 35000, stock: 12, talles: ["s", "m",
"l"] },
  { nombre: "Campera rompevientos", precio: 46000, stock: 2, talles:
["s", "m", "l"] },
];

// Recorrer con forEach() y mostrar solo los productos con stock menor
a 5
console.log("--- 2.b ---");
productos.forEach((producto) => {
  if (producto.stock < 5) {
    console.log(producto);
  }
});

// c. map(): nuevo arreglo con los mismos productos y 10% de descuento
en el precio
console.log("--- 2.c ---");
```

```
{
  const productosConDescuento = productos.map((producto) => {
    return {
      nombre: producto.nombre,
      precio: producto.precio * 0.9,
      stock: producto.stock,
      talles: producto.talles,
    };
  });

  console.log(productosConDescuento);
}

// d. filter(): nuevo arreglo solo con los productos con stock mayor a
10
console.log("--- 2.d ---");
{
  const productosConMuchoStock = productos.filter((producto) =>
producto.stock > 10);

  console.log(productosConMuchoStock);
}
```

Resultado en la consola

Para los puntos 1.a y 1.b se ingresaron por `prompt()` los números 4, 9, 12, 7, 3 y luego 2, 5, 8, 11, 6.

```
--- 1.a ---
[ 4, 9, 12, 7, 3 ]
--- 1.b ---
[ 5, 11 ]
--- 1.c ---
[ 'PERA', 'MANZANA', 'BANANA', 'DURAZNO' ]
--- 1.d ---
[ 'PERA', 'MANZANA', 'BANANA', 'DURAZNO' ]
--- 1.e ---
[ 'manzana', 'banana' ]
--- 2.a ---
{
  nombre: 'Remera básica de algodón',
  precio: 12000,
  stock: 25,
  talles: [ 's', 'm', 'l' ]
}
--- 2.b ---
{
  nombre: 'Buzo con capucha',
  precio: 28000,
  stock: 3,
```

```
  talles: [ 's', 'm', 'l' ]
}
{
  nombre: 'Campera rompevientos',
  precio: 46000,
  stock: 2,
  talles: [ 's', 'm', 'l' ]
}
--- 2.c ---
[
  {
    nombre: 'Remera básica de algodón',
    precio: 10800,
    stock: 25,
    talles: [ 's', 'm', 'l' ]
  },
  {
    nombre: 'Buzo con capucha',
    precio: 25200,
    stock: 3,
    talles: [ 's', 'm', 'l' ]
  },
  {
    nombre: 'Jean recto',
    precio: 31500,
    stock: 12,
    talles: [ 's', 'm', 'l' ]
  },
  {
    nombre: 'Campera rompevientos',
    precio: 41400,
    stock: 2,
    talles: [ 's', 'm', 'l' ]
  }
]
--- 2.d ---
[
  {
    nombre: 'Remera básica de algodón',
    precio: 12000,
    stock: 25,
    talles: [ 's', 'm', 'l' ]
  },
  {
    nombre: 'Jean recto',
    precio: 35000,
    stock: 12,
    talles: [ 's', 'm', 'l' ]
  }
]
```